

Hexa Force: 4-Stage Attack Architecture

This document breaks down the complete cascading failure of container isolation across four distinct stages, leading from initial compromise to physical host takeover.

Hexa Force: Cascading Container Escape Architecture

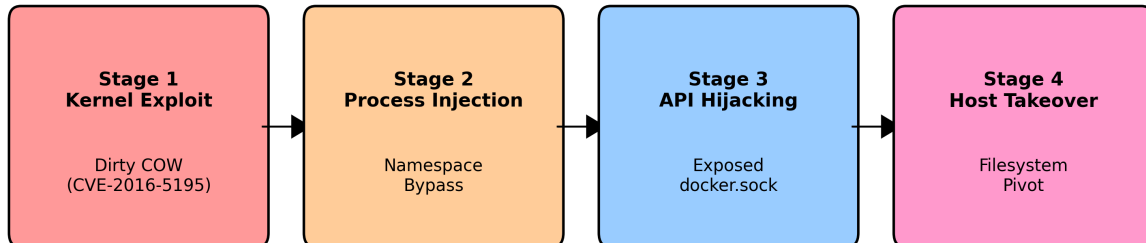


Figure 1: Visual Architecture Flow generated using Matplotlib

Stage 1: The Dirty COW Race Condition (CVE-2016-5195)

Vulnerability: Host Kernel Memory Management (Copy-On-Write)

Architecture: The attacker maps a read-only system file into virtual memory. By concurrently calling `madvise(MADV_DONTNEED)` to discard the private memory mapping and writing to `/proc/self/mem`, a race condition occurs in the kernel. The kernel incorrectly writes the malicious payload directly to the physical, read-only file on disk. This grants the attacker root privileges inside the container.

Stage 2: Process Injection (Namespace Bypass)

Vulnerability: Shared PID Namespace & `CAP_SYS_PTRACE` Capability

Architecture: With container root achieved, the attacker leverages the shared PID namespace to “see” processes running on the underlying host. Using the `CAP_SYS_PTRACE` capability, the attacker injects malicious shellcode directly into the memory space of a privileged host daemon (e.g., a logging or orchestration agent). The payload executes in the context of the host process, achieving lateral movement out of the container boundary.

Stage 3: Docker Daemon API Hijacking

Vulnerability: Exposed `/var/run/docker.sock`

Architecture: The attacker discovers that the host’s Docker Unix socket (`/var/run/docker.sock`) is mounted inside the container. This socket is the raw HTTP API for the Docker engine. By sending crafted POST requests to this socket using `curl`, the attacker gains full administrative control over the host’s container orchestrator, allowing them to stop, delete, or spawn new containers at will.

Stage 4: Persistent Host Takeover (Filesystem Pivot)

Vulnerability: Writable Host Mounts & Unconstrained Deployments

Architecture: Using the hijacked API from Stage 3, the attacker commands the Docker engine to spawn a new “sibling” container. Crucially, they configure this new container to mount the physical host’s root directory (/) into the container (e.g., at `/mnt/host`). The attacker then writes a reverse shell script directly into `/mnt/host/etc/cron.d`. The physical host’s cron daemon blindly executes this script as root, granting the attacker permanent, persistent control over the bare-metal server.